

RxJava Summary for Sessions 10 and 11

Student Sheet

Overview

This summary covers the main RxJava types and operators introduced in sessions 10 and 11. RxJava is a library for composing asynchronous and event-based programs using observable sequences and functional composition patterns.

Core RxJava Types

RxJava provides several types for different scenarios:

Observable

An **Observable** is the fundamental building block. It represents a sequence of events that will be pushed to subscribers over time. Observables are *cold* by default, meaning they start producing values only when someone subscribes. Each subscriber gets its own independent sequence of emissions.

Example use case: reading events from a stream, transforming data, or filtering results.

Single

Single is specialized for scenarios where you expect exactly one item or an error. It is commonly used for operations like HTTP requests that return a single response.

Example: fetching a user from a database returns a **Single<User>**.

Maybe

Maybe can emit zero or one item, or fail with an error. It is useful when an operation may or may not produce a result.

Example: querying a database for an optional config value.

Completable

Completable represents a task that only signals completion or error, with no emitted items. It is used for fire-and-forget operations.

Example: writing to a file or deleting a record.

Flowable

Flowable is like **Observable** but with built-in backpressure support. Use it when the producer may emit items faster than the subscriber can consume them.

1 Session 10. RxJava Basics and Core Operators

RxJava works by creating an **Observable** or related type, chaining operators, and finally subscribing to the stream.

Observable creation

The main creation methods used in the session are **just()**, **fromIterable()**, **fromArray()**, **range()**, and **create()**.

```
Observable.just(1, 2, 3)
    .subscribe(System.out::println);

Observable.fromIterable(Arrays.asList(1, 2, 3))
    .subscribe(System.out::println);

Observable.range(1, 3)
    .subscribe(System.out::println);
```

create() is the most flexible option when values must be emitted manually.

```
Observable.create(emitter -> {
    emitter.onNext(1);
    emitter.onNext(2);
    emitter.onNext(3);
    emitter.onComplete();
}).subscribe(System.out::println);
```

Single, Maybe, and Completable

Single emits exactly one item or an error.

```
Single<String> single = Single.just("hello");
single.subscribe(System.out::println);
```

Maybe emits either one item, completes empty, or fails.

```
Maybe<String> maybe = Maybe.just("Ok");
maybe.subscribe(System.out::println);
```

Completable represents a task that only signals completion or error.

```
Completable.fromAction(() -> Thread.sleep(2000))
    .subscribe(() -> System.out.println("Done"));
```

Timing and hot/cold behavior

interval() emits periodically on a scheduler.

```
Observable.interval(500, TimeUnit.MILLISECONDS)
    .subscribe(System.out::println);
```

A cold observable starts producing values for each subscriber independently. A hot observable shares emissions with all current subscribers. The key methods are **publish()**, **connect()**, and **refCount()**.

```
ConnectableObservable<Long> hot = Observable.interval(1, TimeUnit.SECONDS).publish();
hot.subscribe(x -> System.out.println("A: " + x));
hot.connect();
hot.subscribe(x -> System.out.println("B: " + x));
```

map(), flatMap() and related mapping operators

map() applies a one-to-one transformation: each upstream item is converted to exactly one downstream item. It changes the emitted type from 'T' to 'R'.

```
Observable.range(1, 10)
    .map(n -> n * n) // Integer -> Integer (square)
    .subscribe(System.out::println);
```

flatMap() maps each upstream item to an *asynchronous* inner stream (an **ObservableSource**) and flattens the results into a single output stream. Use it when one input produces zero-or-many outputs or when the inner work is asynchronous.

```
Observable.fromIterable(customerIds)
    .flatMap(id -> getOrderIds(id) // returns Observable<Order>
        .subscribeOn(Schedulers.io()))
    .subscribe(System.out::println);
```

Ordering notes: - **flatMap()** may interleave results from different inner streams (no ordering guarantee). - **concatMap()** preserves input order by queuing inner streams and concatenating them. - **switchMap()** switches to the latest inner stream and cancels previous ones (useful for search/autocomplete scenarios).

```
// Preserve order
Observable.fromIterable(ids)
    .concatMap(id -> getOrderIds(id))
    .subscribe(System.out::println);

// Switch to latest (cancel previous)
searchQueries
    .switchMap(q -> api.search(q).toObservable())
    .subscribe(System.out::println);
```

Mapping to Single/Maybe/Completable:

Sometimes an item maps to a **Single**, **Maybe** or **Completable** rather than an **Observable**. RxJava offers specialized flattening operators that avoid manual conversions:

- **flatMapSingle(mapper)**: map item -> **Single<R>** and flatten into **Observable<R>**.
- **flatMapMaybe(mapper)**: map item -> **Maybe<R>** and flatten (skips empty Maybes).
- **flatMapCompletable(mapper)**: map item -> **Completable** and returns a **Completable** (useful for firing parallel tasks and waiting for completion).

Examples:

```
// map to Single and flatten
Observable.fromIterable(ids)
    .flatMapSingle(id -> api.getUserSingle(id)) // api returns Single<User>
    .subscribe(user -> System.out.println(user.name()));

// map to Maybe and flatten (empty maybes are ignored)
Observable.fromIterable(keys)
    .flatMapMaybe(k -> cache.getMaybe(k)) // Maybe<Value>
    .subscribe(value -> System.out.println(value));

// map to Completable and run tasks concurrently
Observable.fromIterable(tasks)
    .flatMapCompletable(task -> Completable.fromAction(() -> doTask(task)))
    .subscribe(() -> System.out.println("All tasks done"));
```

Why use these specialized operators? They avoid extra wrapping/unwrapping and express intent: the inner source returns a Single/Maybe/Completable, not a multi-item Observable.

Practical pattern: "map to async result" then flatten

```
// Bad: map to Single then need to subscribe inside
Observable.fromIterable(ids)
    .map(id -> api.getUserSingle(id)) // produces Observable<Single<User>>
    // you'd need extra flattening here

// Good: flatten in one step
Observable.fromIterable(ids)
    .flatMapSingle(id -> api.getUserSingle(id))
    .subscribe(user -> System.out.println(user.name()));
```

Concrete mapping examples

These short snippets illustrate behavior differences (ordering, cancellation and empty results):

```
// flatMap: inner streams may interleave (no ordering guarantee)
Observable.just(1,2,3)
    .flatMap(n -> Observable.just(n)
        .delay((4 - n) * 100, TimeUnit.MILLISECONDS))
    .subscribe(System.out::println);

// concatMap: preserves input order by concatenating inner streams
Observable.just(1,2,3)
    .concatMap(n -> Observable.just(n)
        .delay((4 - n) * 100, TimeUnit.MILLISECONDS))
    .subscribe(System.out::println);

// switchMap: switches to latest inner stream, cancelling previous ones
Observable.interval(300, TimeUnit.MILLISECONDS)
    .take(4)
    .switchMap(i -> Observable.interval(0, 200, TimeUnit.MILLISECONDS)
        .map(j -> i + ":" + j))
    .take(8)
    .subscribe(System.out::println);

// flatMapSingle: map to Single and flatten
Observable.just(10, 20)
    .flatMapSingle(n -> Single.fromCallable(() -> n * 2)
        .subscribeOn(Schedulers.computation()))
    .subscribe(System.out::println);

// flatMapMaybe: map to Maybe; empty Maybes are skipped
Observable.range(1, 6)
    .flatMapMaybe(n -> n % 2 == 0 ? Maybe.just(n) : Maybe.empty())
    .subscribe(System.out::println);

// flatMapCompletable: run tasks for each item and wait for completion
Observable.range(1, 4)
    .flatMapCompletable(n -> Completable.fromAction(() -> System.out.println("task " +
        n))
        .subscribeOn(Schedulers.io()))
    .subscribe(() -> System.out.println("All tasks done"));
```

filter()

filter() keeps only items that satisfy a condition.

```
Observable.range(1, 100)
    .filter(RxJavaUtils::isPrime)
    .subscribe(System.out::println);
```

distinct()

distinct() removes repeated elements.

```
Observable.just(1, 2, 2, 3, 3, 3)
    .distinct()
    .subscribe(System.out::println);
```

takeWhile()

takeWhile() keeps emitting until the predicate becomes false.

```
Observable.range(1, 100)
    .takeWhile(n -> n <= 50)
    .subscribe(System.out::println);
```

2 Session 11. Combining, Scheduling, Error Handling, and Backpressure

This session goes deeper into reactive pipelines and adds time-based and backpressure-aware operators.

merge()

merge() combines several observables into one stream and interleaves their emissions as they arrive.

```
Observable.merge(names, colors)
    .subscribe(System.out::println);
```

zip()

zip() pairs emissions position by position.

```
Observable.zip(names, grades, Grade::new)
    .subscribe(System.out::println);
```

groupBy()

groupBy() splits one stream into groups by key.

```
Observable.fromIterable(words)
    .groupBy(word -> word.substring(0, 1))
    .subscribe(group -> System.out.println(group.getKey()));
```

reduce()

reduce() accumulates all items into a single result.

```
Observable.range(1, 10)
    .reduce(0, Integer::sum)
    .subscribe(System.out::println);
```

subscribeOn() and observeOn()

subscribeOn() decides where the upstream work starts, while **observeOn()** changes the thread used downstream.

```
Observable.fromCallable(() -> heavyComputation())
    .subscribeOn(Schedulers.io())
    .observeOn(Schedulers.computation())
    .subscribe(System.out::println);
```

retry()

retry() resubscribes automatically after an error.

```
Observable.fromCallable(() -> riskyOperation())
    .retry(3)
    .subscribe(System.out::println, Throwable::printStackTrace);
```

onErrorResumeNext()

onErrorResumeNext() switches to a fallback observable when an error occurs.

```
source.onErrorResumeNext(Observable.just(10, 11, 12))
    .subscribe(System.out::println);
```

buffer()

buffer() collects emissions into lists before sending them downstream.

```
Observable.interval(100, TimeUnit.MILLISECONDS)
    .buffer(1, TimeUnit.SECONDS)
    .subscribe(batch -> System.out.println(batch.size()));
```

window()

window() also groups values over time, but it emits each group as another observable.

```
Observable.interval(100, TimeUnit.MILLISECONDS)
    .window(1, TimeUnit.SECONDS)
    .subscribe(window -> window.count().subscribe(System.out::println));
```

debounce()

debounce() waits for a pause before letting a value through, which is useful to remove noisy intermediate emissions.

```
keyStrokes
    .debounce(50, TimeUnit.MILLISECONDS)
    .subscribe(System.out::println);
```

sample()

sample() periodically takes the latest value seen so far.

```
keyStrokes
    .sample(50, TimeUnit.MILLISECONDS)
    .subscribe(System.out::println);
```

Flowable and backpressure

When the producer is faster than the consumer, **Flowable** can be used instead of **Observable**.

```
Flowable.interval(1, TimeUnit.NANOSECONDS)
    .onBackpressureDrop()
    .observeOn(Schedulers.computation())
    .subscribe(System.out::println);
```

onBackpressureDrop() discards items that cannot be delivered in time, while **onBackpressureLatest()** keeps only the most recent item.

```
source.onBackpressureLatest()
    .subscribe(System.out::println);
```

take()

take() limits the number of items emitted or the time window for which emissions occur.

```
Observable.range(1, 100)
    .take(10)
    .subscribe(System.out::println);

// Or take for a duration
Observable.interval(100, TimeUnit.MILLISECONDS)
    .take(5, TimeUnit.SECONDS)
    .subscribe(System.out::println);
```

count()

count() counts the number of items emitted by the observable and returns a **Single** with the count.

```
Observable.range(1, 10)
    .count()
    .subscribe(count -> System.out.println("Total: " + count));
```

max() and maxBy()

max() emits the maximum item from a stream, while **maxBy()** allows specifying a key selector.

```
Observable.just(5, 2, 8, 1, 9)
    .max(Integer::compareTo)
    .subscribe(System.out::println);

// maxBy with a key extractor
Observable.range(1, 10)
    .maxBy(n -> n * n)
    .subscribe(System.out::println);
```

average()

average() computes the arithmetic mean of the emitted numbers.

```
Observable.just(1, 2, 3, 4, 5)
  .average()
  .subscribe(avg -> System.out.println("Average: " + avg));
```

throttleFirst() and throttleLast()

throttleFirst() emits the first item in each time window and drops the rest. **throttleLast()** emits the last item in each window. These are useful to avoid false alarms from rapid successive events.

```
// Only emit the first event in each 2-second window
events
  .throttleFirst(2, TimeUnit.SECONDS)
  .subscribe(System.out::println);

// Or emit the last event in each window
events
  .throttleLast(2, TimeUnit.SECONDS)
  .subscribe(System.out::println);
```

Mapping Cheat-sheet

One-line reference for common mapping operators:

- **map**: $T \rightarrow R$, synchronous one-to-one transform.
- **flatMap**: $T \rightarrow \text{ObservableSource}\langle R \rangle$, flattens inner streams; concurrency and interleaving allowed.
- **concatMap**: like **flatMap** but preserves input order (concatenates inner streams).
- **switchMap**: switches to the latest inner stream, cancelling previous ones.
- **flatMapSingle**: $T \rightarrow \text{Single}\langle R \rangle$, flattens **Single** into main stream.
- **flatMapMaybe**: $T \rightarrow \text{Maybe}\langle R \rangle$, flattens **Maybe** and skips empty results.
- **flatMapCompletable**: $T \rightarrow \text{Completable}$, runs **Completable**s and returns a **Completable** representing their completion.
- **map to Single/Maybe/Completable pattern**: prefer specialized flattening operators to avoid nested wrapper types.

Quick Reference

- **Types**: Use **Observable** for general streams, **Single** for single values, **Maybe** for optional values, **Completable** for completion-only tasks, and **Flowable** for backpressure-aware streams.
- **Creation**: **just()**, **fromIterable()**, **fromArray()**, **range()**, **create()**, and **interval()** are the main methods to create streams.

- **Transformation / Mapping:** `map()` for one-to-one transforms; `flatMap()`, `concatMap()`, and `switchMap()` for mapping to inner streams (flattening). Use `flatMapSingle()`, `flatMapMaybe()` and `flatMapCompletable()` when the inner source is a `Single`, `Maybe` or `Completable` respectively.
- **Filtering:** `filter()`, `distinct()`, `takeWhile()`, `take()`, `throttleFirst()`, and `debounce()`.
- **Aggregation:** `reduce()`, `count()`, `max()`, `average()`, and `groupBy()`.
- **Combination:** `merge()` for interleaving, `zip()` for pairing position-by-position.
- **Batching:** `buffer()` collects items into lists, `window()` emits them as observables, `sample()` periodically takes the latest value.
- **Scheduling:** `subscribeOn()` decides where upstream work runs, `observeOn()` changes the thread downstream.
- **Error Handling:** `retry()` resubscribes on error, `onErrorResumeNext()` switches to a fallback observable.
- **Backpressure:** `onBackpressureDrop()` discards items, `onBackpressureLatest()` keeps the most recent.